Contexto de la Problemática (Para el alumno)

Estás diseñando el módulo de alertas de una plataforma financiera internacional. El sistema debe procesar eventos críticos (como transferencias sospechosas o alertas de inicio de sesión) y despacharlos al usuario.

El negocio exige máxima flexibilidad:

- Un mensaje básico siempre se procesa de forma plana en el sistema.
- Dependiendo de la urgencia del evento o de las preferencias del cliente, el sistema debe poder acoplar dinámicamente canales adicionales de salida: enviar un **SMS**, disparar un correo electrónico (**Email**), o enviar una alerta en tiempo real a la aplicación móvil (**Push Notification**).
- Además de los canales, existen capas de formato de negocio: por ejemplo, agregar una firma digital de cumplimiento legal o aplicar una capa de urgencia que altere la prioridad del mensaje.

Tu tarea es construir este motor utilizando el patrón **Decorator**, permitiendo que una alerta base sea "envuelta" por diferentes canales y capas de formato según las necesidades del momento.

📋 Consignas y Pasos del Ejercicio:

Paso 1: El Paquete de Datos y el Contrato (15 puntos)

1. Crear una clase de soporte llamada PaqueteMensaje que transporte el estado del envío. Debe contener los atributos públicos: contenido (String), destinatario (String) y un listado o banderas booleanas de canales activos (enviaSMS, enviaEmail, enviaPush).
2. Definir la interfaz raíz Notificacion (el Componente) con un único método crítico: void procesarYEnviar(PaqueteMensaje paquete).

Paso 2: El Componente Concreto (15 puntos)

1. Crear la clase AlertaBase que implemente Notificacion.
2. Definir su comportamiento por defecto: al ejecutarse procesarYEnviar, simplemente debe imprimir en la consola del servidor un registro del sistema que diga: "[LOG SISTEMA] Registrando alerta base para el usuario: " + paquete.destinatario. Este componente no envía nada por sí solo, solo inicia el flujo de procesamiento.

Paso 3: La Infraestructura del Patrón (20 puntos)

1. Implementar la clase abstracta DecoradorNotificacion que implemente Notificacion y mantenga una referencia protegida (# notificacionEnvuelta) a través de composición.
2. Crear tres clases abstractas intermedias especializadas que hereden de ella para clasificar las ranuras de infraestructura: DecoradorCanal, DecoradorFormato y DecoradorSeguridad.

Paso 4: Implementación de Capas Reales de Infraestructura (35 puntos)

Desarrollar los siguientes decoradores concretos sobreescribiendo los flujos correspondientes:

1. **EnvioSMS (Hereda de DecoradorCanal):** Debe llamar a la capa interna y luego inyectar la simulación del envío de mensajería celular: paquete.enviaSMS = true; System.out.println("[SMS] Despachando mensaje de texto a las antenas móviles...");.
2. **FirmaLegal (Hereda de DecoradorFormato):** Debe interceptar el paquete y modificar el atributo contenido, concatenándole al final un texto fijo de términos y condiciones de la empresa antes de pasarlo a la siguiente capa.
3. **CanalHíbridoPushEmail (Hereda de DecoradorSeguridad):** Un componente de conectividad mixto. Debe alterar el comportamiento de forma simultánea activando las banderas enviaPush = true y enviaEmail = true, además de simular el envío por las APIs de Google Firebase y SMTP de forma paralela.

Paso 5: Simulación del Entorno de Producción (15 puntos)

En la clase principal Main, simular el comportamiento de una fintech:

1. Instanciar una AlertaBase para un usuario ficticio.
2. Envolver dinámicamente esa alerta simulando un evento de fraude financiero crítico: aplicar la FirmaLegal, luego el canal EnvioSMS y finalmente el CanalHíbridoPushEmail.
3. Ejecutar el método procesarYEnviar sobre la envoltura más externa y analizar cómo el mensaje va mutando y disparando las distintas APIs en cascada a través de las capas de software.

- *Diagrama U.M.L:*

